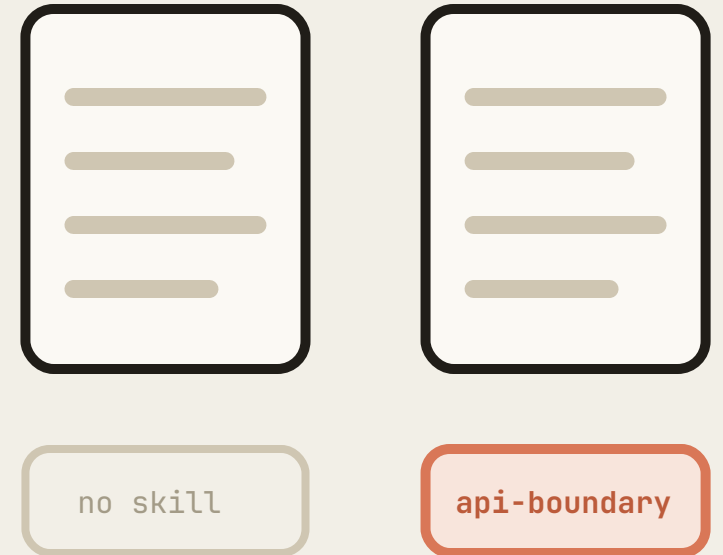


# Nothing to build. Everything to read.

Two completed apps from the same spec. One was written with the skill loaded, one without. Same reviewer, same rules, one variable.

```
> /review-build AssetDesk • /review-build reference/AssetDesk-noskills
```



# You have watched one bucket empty.

## CHECKPOINT 1 • THE SPEC ALONE

A complete, working app, and roughly ten findings. About half clustered around one architectural choice the spec declined to make.

## CHECKPOINT 2 • PLUS ONE SKILL

Sixty-nine lines added to `.claude/`. The architecture cluster went to zero. The quality findings did not move, because nothing addressed them.

## WHAT IS LEFT

A handful of defects that have survived both builds. They are the subject of checkpoint 4, but first you are going to read them yourself, in code you did not write.

## WHY THERE IS NO BUILD HERE

Both apps are already committed on this branch. Nothing to wait for, which is the point: this is the one checkpoint where **you** do the work rather than watching.

A bare `dotnet build` now fails, because the branch holds two projects. Name the one you mean.

## READING BEATS WATCHING

Reviewing generated code is the skill that actually transfers to Monday. You will not be writing much of it. You will be deciding whether to accept it.

# Identical spec. Identical defects. **Different shape.**

checkpoint-3 · both directories build with 0 warnings and seed to 5 employees, 12 assets

+ **AssetDesk/** **written with api-boundary** · typed client, DTOs, cancellation, one error translation

- **reference/AssetDesk-noskills/** · components inject the repository directly, no HTTP layer

= the same five quality defects, planted identically in both that is the control

## WHAT MAKES THIS A CONTROL, NOT A DEMO

If the two apps differed in architecture **and** in quality, you could not attribute anything. Because the five quality defects are byte-for-byte the same in both, every difference the reviewer reports is architectural by construction.

## PROVE IT TO YOURSELF IN ONE COMMAND

```
diff -rq AssetDesk reference/AssetDesk-noskills
```

The files that differ are the components, Program.cs, and two new files in Data/. Nothing else. That list is the boundary.

# A defect is only a finding if **a rule names it.**

## THIS BRANCH SHIPS ONE RULEBOOK, NOT THREE

ls .claude/skills here gives you api-boundary and review-build. The two quality skills **do not exist yet**. They arrive at checkpoint 4.

So the reviewer's QUALITY bucket reads **0 on both apps**, while the defects are sitting right there in the code. It is not missing them; it was never given the rule.

## WHERE THEY SURFACE INSTEAD

Three of the five also break a SPEC.md §9 checkbox, so they land in the SPEC bucket: the swallowed message, the bypassed guard, and the collapsed component §3.2 names.

The other two, the suppressed warning and the magic string, **go unreported here entirely**.

## THE SAME DEFECT, TWO DIFFERENT BUCKETS

The swallowed exception is a SPEC finding today and a QUALITY finding at checkpoint 4. The code never changed. **The rulebook did.**

## WHAT THIS MEANS FOR ANY AUDIT

Your audit is exactly as good as the rules you handed it. A clean report means **"nothing I was told to look for"**, never "nothing wrong".

Which is why the reviewer is forbidden from inventing standards. A reviewer that improvises makes every branch score the same, and the experiment dies.

# Five rules the reviewer follows. Borrow them.

## 1 · POINT AT A LINE, OR SAY NOTHING

A file and a line number. "Consider whether the error handling is robust" is not a finding. It is a feeling, and it cannot be argued with or fixed.

## 2 · CITE THE RULE, NOT YOUR TASTE

Head every finding with its source: SPEC.md §4.6, or a skill name. If you cannot name the rule, you are stating a preference rather than reviewing.

## 3 · ABSENCE IS NOT A DEFECT

No tests, no interfaces, no service layer, no CI. All **correct here**, because §2 forbids them. Flagging them means you are reviewing the spec, not the build.

## 4 · ONE DECISION, ONE FINDING

Five components injecting the repository is **one** finding with five locations. Report it five times and the architecture noise buries everything else.

## 5 · DO NOT FIX ANYTHING

The reviewer has no write tools at all. Separating "find" from "fix" is deliberate: the moment you start fixing, you stop looking.

## WHERE TO START READING

Not at the top. Start at `Data/AssetRepository.cs`, the SQL and the transactions, then `Program.cs`, then the components. Data first, same order the spec built it.

Two agents, two windows, one per directory, so neither review contaminates the other's context.

# A red banner with **nothing** in it.

```
Components/Shared/AssignDialog.razor
```

```
catch (AssetDeskException)
{
    _error = ""; // the message is gone
}
```

and six lines up, in the markup:

```
@if (_error is not null) { <render the red box> }
```

`""` is not `null`. So the guard passes, the box renders, and the reason is discarded.

## REPRODUCE IT IN THIRTY SECONDS

Run either app. Mark an asset **In repair**, then try to assign it. The repository refuses with `AST-1004 is in repair and cannot be assigned.`, and the UI shows you an empty red rectangle.

## WHY IT IS THE WORST KIND OF BUG

It compiles. It has a test-shaped happy path. Nothing logs. The user sees that something failed and is told **nothing about what**, so they retry, identically, forever.

## AND IT BREAKS THE SPEC IN WRITING

§4.6: the UI shows `ex.` Message verbatim and never writes its own copy. §9 makes it a checkbox. This is a SPEC finding today, before any quality skill exists.

# What survived two builds.

|   | THE DEFECT                           | WHY IT IS REAL                             | REPORTED HERE AS |
|---|--------------------------------------|--------------------------------------------|------------------|
| 1 | The swallowed exception              | User told nothing; §4.6 broken             | SPEC             |
| 2 | A <NoWarn> hiding a real null deref  | Build looks clean; crash is still there    | not reported     |
| 3 | The server-side guard, commented out | curl bypasses validation entirely          | SPEC             |
| 4 | The table inlined into Home.razor    | 300 lines, four jobs, a duplicate row type | SPEC             |
| 5 | Status compared as a string literal  | Rename the enum, count silently hits zero  | not reported     |

Two of the five are invisible to today's audit. **At checkpoint 4 all five become findings, and not one line of the code will have changed.**

# The shape changed. The quality **did not**.

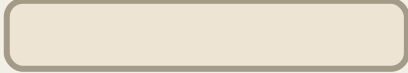
**Fair to conclude:** one skill removed one entire class of finding, repeatably, with no change to the spec or the prompt. **Not fair to conclude:** that the api-boundary app is better software. It carries the same five defects, and it costs an extra hop and a duplicated shape to get them.

BOUNDARY

 no skill

 0 · with the skill

THE FIVE DEFECTS

 no skill

 with the skill

unchanged, on purpose

---

checkpoint 4 →

- + a rule you write down once stops being a rule you have to remember
- and a rule you never wrote down was never enforced, by anyone, ever